

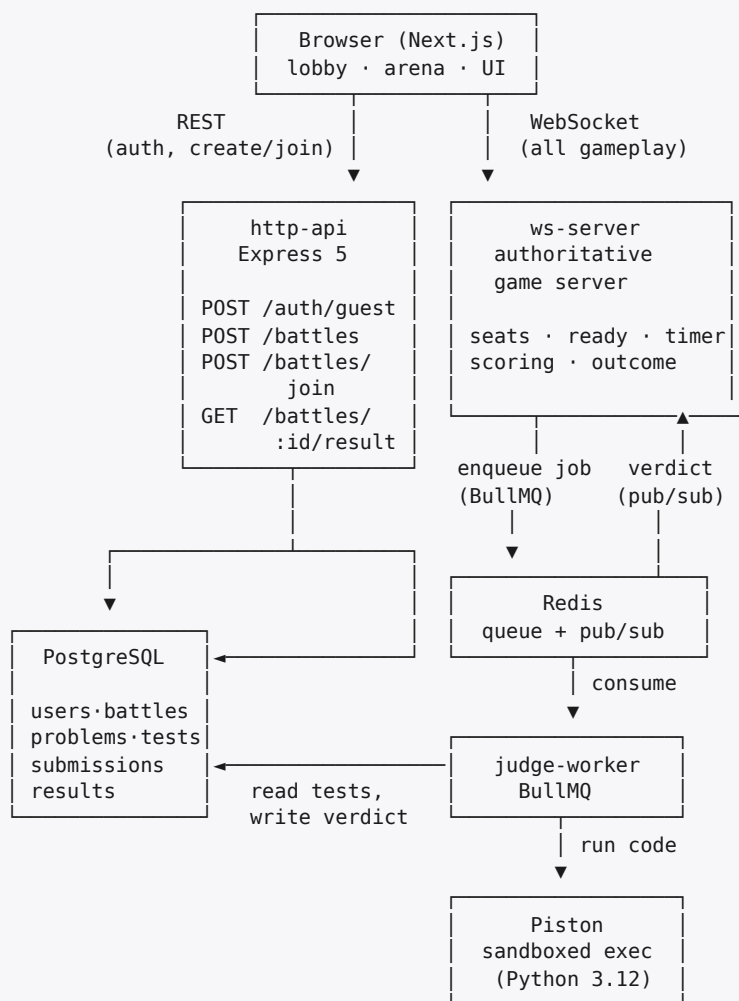
Technical Approach & Architecture

CombatX — real-time competitive coding battles

Document 1 of 3 · [Why/What/How](#) · [User Guide](#)

1. System overview

CombatX is a pnpm/Turborepo monorepo containing four runtime services and five shared packages. The browser holds two connections: REST to `http-api` for transactional work, and a long-lived WebSocket to `ws-server` for the battle.



Service responsibilities

Service	Owns	Explicitly does <i>not</i>
<code>web</code>	Rendering, input, optimistic UI	Any authoritative state
<code>http-api</code>	Guest auth, battle creation/join, result reads	Live gameplay
<code>ws-server</code>	All game state: seats, readiness, timer, scoring, outcome	Executing code
<code>judge-worker</code>	Running submissions, producing verdicts	Deciding who wins

The separation between the last two is the core design decision: **the referee never executes code, and the executor never decides outcomes.**

2. The authority model

Every piece of contestable state lives on `ws-server`. Clients send *intent*; the server decides what actually happened.

Client sends	Server decides
<code>team:select</code>	Whether that seat is free and the battle is open
<code>player:ready</code>	Whether the lobby is now startable
<code>battle:start</code>	Whether the requester is the host and rules pass
<code>code:submit</code>	The submission's server-side timestamp

A client cannot mark itself ready, award itself a point, set its own submission time, or end a battle. There is no client-supplied timestamp anywhere in the scoring path — `submittedAt` is stamped on receipt by the server and is, per `packages/game/src/outcome.ts`, *"the sole tie-break authority."*

Why this matters for scoring

The outcome rule is: **highest passed-count wins; ties break by who reached that count first.** If clients supplied their own timestamps, every tie would be winnable by lying. Server-stamped receipt time closes that.

Deterministic problem assignment

Problems are not chosen randomly per battle. `battleRoom.ts` selects from the candidates for the room's difficulty using **`battleId` as the seed**, so the assignment is reproducible: every participant provably receives the same problem, and a battle can be replayed or audited without storing the choice separately.

3. Wire protocol

All frames are Zod-validated in both directions (`packages/protocol`). An invalid frame is rejected, not coerced.

Client → server: `hello` · `team:select` · `player:ready` · `battle:start` · `code:submit` · `battle:leave` · `ping`

Server → client: `snapshot` · `lobby:update` · `presence:update` · `battle:countdown` · `battle:start` · `timer:tick` · `submission:queued` · `submission:result` · `opponent:progress` · `battle:finished` · `ack` · `ack-error` · `error` · `pong`

The opponent-safety boundary

`opponent:progress` is documented in `packages/protocol/src/messages.ts` as *"The ONLY progress signal opponents receive (no source)."* It carries a passed-count and nothing else — no code, no test names, no failure output.

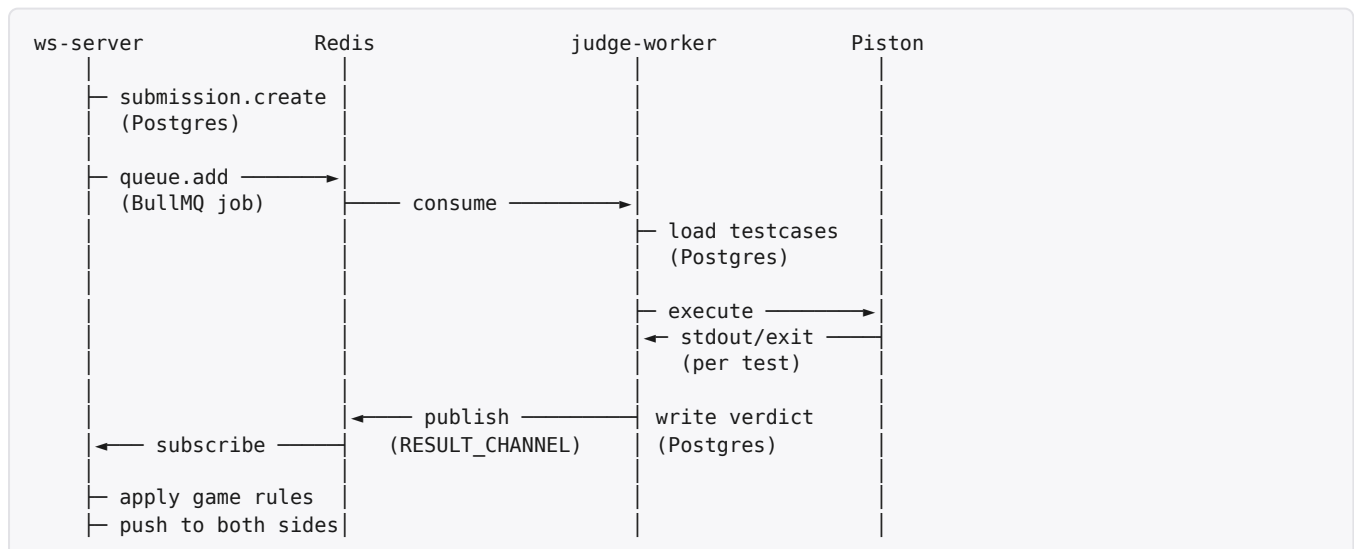
This is enforced by the schema, not by convention. There is no field on the opponent-facing message that *could* carry source, so a future change that tries to leak it fails type-checking in both the client and the server.

Reconnection

`snapshot` carries full current state, letting a client that drops mid-battle rebuild without special-case recovery logic. The server is the single source of truth, so replaying state is just sending it again.

4. The judging pipeline

Executing untrusted code is both **dangerous** and **slow**. Those are two different problems with two different solutions.



Danger → **isolation**. Code runs in a Piston container, never in the app processes. `judge-worker` is the only service that talks to it.

Slowness → **decoupling**. Execution takes hundreds of milliseconds per test. A game server that blocks on that stops sending timer ticks to *everyone in the room*. So submissions become BullMQ jobs; the socket path stays hot regardless of judge load, and workers scale independently (`JUDGE_CONCURRENCY`).

Defensive limits

`apps/judge-worker/src/judge/piston.ts` applies:

- `run_timeout` / `compile_timeout` — passed to Piston per execution
- an `AbortController` on the HTTP call (`httpTimeoutMs`) so a hung sandbox can't wedge a worker
- **output truncation** — commented in-source as ensuring *"one runaway print can't blow up memory"*

Jobs use `jobId: submissionId` , making enqueue idempotent: a retry or duplicate frame cannot double-judge one submission.

5. Data model

Eight Prisma models (`packages/db/prisma/schema.prisma`):

```
graph LR
    User <-- TeamMember >-- Team <-- Battle
    Problem <-- TestCase
    Battle --> Submission
    Submission --> Result
```

Model	Role
<code>User</code>	Guest identity. No password field — guest-only
<code>Problem</code>	Statement and metadata
<code>TestCase</code>	Hidden inputs/outputs. Read only by <code>judge-worker</code>
<code>Battle</code>	Room, mode, lifecycle state, timing
<code>Team</code>	Side A or B within a battle
<code>TeamMember</code>	Seat: which user on which team
<code>Submission</code>	One attempt, with server-stamped <code>submittedAt</code>
<code>Result</code>	Final standings and deciding submission

Battle lifecycle: `LOBBY` → `IN_PROGRESS` → `FINISHED` .

Guest-only play is **structural, not policy** — there is no credential column to misuse, so there is no password to leak.

6. Type and validation strategy

Two shared packages are compiled into both the frontend and the backend:

- `packages/protocol` — Zod schemas for every REST body and socket frame
- `packages/game` — pure functions for lobby readiness, scoring, and outcome

This is what prevents client/server drift. A protocol change that breaks a client surfaces as a **type error at build time**, not as a mystery bug during someone's tournament.

Keeping the rules pure also makes the part that decides winners the part that's easiest to verify:

`packages/game` is unit-tested in isolation, no database, no sockets (9 tests in `outcome.test.ts`).

Validation is applied at *every* boundary — REST bodies, socket frames in both directions, and jobs pulled off the queue (`JudgeJob.parse`). Nothing is trusted because it arrived over a channel that was trusted a moment ago.

7. Deployment

Each app ships two Dockerfiles:

Variant	Build	Purpose
<code>Dockerfile.dev</code>	3 stages, source bind-mounted	Hot reload
<code>Dockerfile.prod</code>	5–6 stages, <code>turbo prune</code> , non-root user	Slim, reproducible runtime

`turbo prune` isolates each app's dependency subtree, so an unrelated app's change doesn't invalidate the install cache. The result is measurable — `web` is **331 MB** in prod versus **1.19 GB** in dev.

One-command startup

Two init containers make `docker compose up` sufficient:

- `db-init` — applies the schema, seeds problems
- `piston-init` — installs the Python 3.12 runtime into the sandbox

Both are **idempotent** (the provisioner checks the installed-runtime list before POSTing) and both are gated behind healthchecks, so no app service can start against an unmigrated database or an empty sandbox.

8. Known constraints

- **1v1 is what ships.** 2v2–4v4 exist in schema, protocol, and rules, but are gated in the UI pending a larger-roster lobby flow.

- **Python only.** Adding a language means adding it to `PISTON_RUNTIME` in `packages/protocol/src/enums.ts` *and* the provisioner list — they must match.
- **Single ws-server instance.** Battle state is in-process. Horizontal scaling would need room affinity or state extraction to Redis.
- **No rate limiting on submissions** beyond queue concurrency.